

Lab 1: AI Usage Transcript

Todo App (Next.js + SQLite) — COMS3011A Software Design

AI Declaration: This document is an unedited transcript of the AI-assisted development session for Lab 1, conducted using **Claude-Web**[**Claude Sonnet 5**]. The transcript below has been compiled and formatted (headings, speaker labels, and phase categorisation added for readability) with the assistance of the same tool. No content or meaning has been altered from the original session.

Per the module AI policy, each exchange below is labelled with the phase of work it belongs to: **Planning**, **Code Generation**, or **Debugging**.

Turn 1 — Planning

You:

For our third year Computer Science module, software design project, in our first lab(lab 1), we've been tasked with building a todo app using Next.js and SQLite. It is not to be deployed to the web and is to be ran locally via Node.js npm or equivalent. I've never used Next.js or SQLite so where do I start?

Claude:

Great combo for a first project — Next.js gives you both frontend and backend in one app, and SQLite is about as low-friction as databases get. Here's how I'd approach it.

The mental model first

Next.js is a React framework. For your purposes, the two pieces that matter are:

- **Pages/UI** — React components that render your todo list
- **API routes / Server Actions** — server-side code (runs in Node, not the browser) where you can talk to a database

SQLite is a database that lives in a single file on disk (e.g. `todos.db`) — no server process to run, no accounts to set up. Perfect for “runs locally” requirements like yours.

Since Next.js runs on Node.js, and SQLite is just a file, your database connection code lives in your API routes/Server Actions (server-side only — never import a SQLite driver into a component that runs in the browser).

Setup steps

1. Scaffold the app

```
npx create-next-app@latest todo-app  
cd todo-app
```

Say yes to TypeScript if you're comfortable with it (recommended, catches bugs early), and yes to the App Router when prompted (this is the current standard).

2. Add a SQLite driver The simplest option for a beginner, local-only project is `better-sqlite3` — it's synchronous (no `async/await` juggling), fast, and has a very small API:

```
npm install better-sqlite3
```

3. Create a database module Something like `lib/db.ts`:

```
import Database from 'better-sqlite3';

const db = new Database('todos.db');

db.exec(`
  CREATE TABLE IF NOT EXISTS todos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    completed INTEGER DEFAULT 0
  )
`);

export default db;
```

This runs once, creates the table if it doesn't exist, and gives you a `db` object to query elsewhere.

4. Build your backend logic Two common patterns in the App Router:

- **Route Handlers** (`app/api/todos/route.ts`) — classic REST-style endpoints (GET, POST, etc.) that your frontend fetches from
- **Server Actions** — functions marked `'use server'` that your React components can call directly, no manual fetch needed

For a first Next.js project, I'd actually recommend starting with **Route Handlers** — they map more directly onto the request/response model you may already know from other courses, and debugging is more intuitive. Server Actions are a bit more “magic” and easier to reach for once you're comfortable.

5. Wire up the UI Fetch todos in a Server Component (runs on the server, can query `db` directly or call your API), and use a small Client Component (`'use client'`) for the interactive bits — the input box, checkboxes, delete buttons — since those need browser event handlers.

A sensible order of operations for lab 1

1. Get `create-next-app` running (`npm run dev`, confirm you see the default page at `localhost:3000`)
2. Add `better-sqlite3`, create the table, write one script to confirm you can insert/read a row

3. Build a GET `/api/todos` route that returns all todos as JSON
4. Build a page that fetches and displays them
5. Add POST (create), then PATCH/PUT (toggle complete), then DELETE
6. Hook the UI buttons/forms up to those routes

A couple of things worth knowing upfront

- `better-sqlite3` needs to compile a native module on install — if that fails, it's usually a missing build toolchain (Python + C++ compiler); worth testing the install early rather than the night before it's due.
- Keep `todos.db` out of git (add it to `.gitignore`) unless your assignment wants seed data committed.
- Don't import `better-sqlite3` in a Client Component — it'll break, since it's a Node-only module and Client Components can also run in the browser.

Want me to scaffold a minimal working version of this (project structure, db module, one API route, one page) so you have a running skeleton to build from?

Turn 2 — Planning

You:

are these defaults fine? TypeScript, ESLint, No React Compiler, Tailwind CSS, No `src/` directory, App Router, AGENTS.md

Claude:

Yes, those are all sensible for this project — let me go through them quickly:

- **TypeScript** — good. Catches typos/type mismatches early, and gives you autocomplete for your data shapes (e.g. what fields a `Todo` has). Slightly more upfront syntax but worth it.
- **ESLint** — good, just a linter flagging obvious mistakes as you code. Not deployed, so it's low-stakes, but no reason to turn it off.
- **No React Compiler** — fine to leave off. It's a newer optimization feature, not something a todo app needs, and one less variable if something breaks.
- **Tailwind CSS** — good default. Faster than writing separate CSS files for a small app, styles live right in your JSX.
- **No `src/` directory** — fine either way, purely organizational. Without it, your `app/` folder sits at the project root instead of inside `src/`. For a small project like this, skipping it keeps paths shorter.
- **App Router** — yes, this is what my earlier steps assumed (Route Handlers in `app/api/.../route.ts`).
- **AGENTS.md** — this one's worth a flag. It's a file `create-next-app` can generate to give AI coding assistants (like Claude Code) context about your project conventions.

It's harmless to include, but it doesn't affect your app's functionality at all — it's not something your lecturer/marker will care about either way. Include it if you want, ignore it otherwise.

One thing to double check: since your module cares about you learning Next.js + SQLite from scratch, if the brief says anything about not using AI coding agents to generate the project, the AGENTS.md file is irrelevant either way — it only matters if you're using an AI agent tool to write code, not if you're coding by hand with Claude as a helper/explainer (which is what we're doing here).

Go ahead with those defaults — they match everything in my setup steps above.

Turn 3 — Planning

You:

for a github repo, if ive already created the project, do I still run these? `echo "# SDLab1-
ToDo" >> README.md git init git add README.md git commit -m "first commit" git branch
-M main git remote add origin https://github.com/gara-codes/SDLab1-ToDo.git git push -u
origin main`

Claude:

Not quite as-is — that snippet assumes you're starting from an empty folder with nothing but a README. Since you already ran `create-next-app` and have a real project with files, adjust it like this:

```
cd todo-app          # if you're not already inside it

git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/gara-codes/SDLab1-ToDo.git
git push -u origin main
```

Key differences from GitHub's snippet:

- **Skip the `echo "# SDLab1-ToDo" >> README.md` line** — `create-next-app` already generated a README (and everything else). No need to create a new one; you'd just be overwriting/duplicating.
- **Use `git add .` instead of `git add README.md`** — you want to stage your whole project (the app/ folder, package.json, etc.), not just a README.

One important check before you commit: `create-next-app` should have already generated a `.gitignore` file for you (worth confirming it exists — `cat .gitignore` or check in your file explorer). It should include `node_modules/` at minimum. Since you're also using SQLite, add your database file to it too:

```
# add this line to .gitignore
todos.db
```

You don't want `node_modules/` (huge, regenerable via `npm install`) or your SQLite database file (local data, not code) ending up in the repo.

Quick sanity check before pushing:

```
git status
```

This shows you what's about to be committed — worth a glance to confirm `node_modules/` isn't in the list (if it is, your `.gitignore` isn't catching it, likely because it wasn't in place before your first `git add`).

Once that all looks right, the rest of your sequence (`init`, `add`, `commit`, `branch -M main`, `remote add origin`, `push -u origin main`) is correct and only needs to run once for this repo.

Turn 4 — Code Generation

You:

Build a `GET /api/todos` route that returns all todos as JSON

Claude:

Here's the Route Handler for that. In the App Router, API routes live in `app/api/<name>/route.ts` — the folder path becomes the URL path, and `route.ts` is a reserved filename Next.js looks for.

1. Create the file `app/api/todos/route.ts`:

```
import { NextResponse } from 'next/server';
import db from '@/lib/db';

export async function GET() {
  const todos = db.prepare('SELECT * FROM todos ORDER BY id DESC').all();
  return NextResponse.json(todos);
}
```

A few things to note:

- **`export async function GET()`** — Next.js maps this name directly to HTTP GET requests hitting `/api/todos`. If you later add `export async function POST()` in the same file, that handles POST requests to the same URL.
- **`db.prepare(...).all()`** — this is better-sqlite3's pattern: `.prepare()` compiles the SQL statement, `.all()` runs it and returns every matching row as an array of plain objects. (There's also `.get()` for a single row, and `.run()` for inserts/updates/deletes — you'll use those in the next routes.)

- **ORDER BY id DESC** — optional, but usually nicer so newest todos show first. Drop it if you'd rather keep insertion order.
- **NextResponse.json(...)** — wraps your data in a proper JSON HTTP response with the right headers, so the frontend can `fetch()` it and call `.json()` on the result.
- **@/lib/db** — this assumes the `@/` import alias is set up, which `create-next-app` configures by default when you don't use a `src/` folder. If that import fails, use a relative path instead, e.g. `../..../lib/db` (adjust `../` count based on your folder depth) — let me know if you hit that and I'll help sort the path out.

2. Test it

With your dev server running (`npm run dev`), visit:

`http://localhost:3000/api/todos`

You should see a JSON array in the browser — `[]` if your table's empty, or your rows if you inserted test data earlier.

If you haven't inserted any test rows yet and want to sanity-check the whole pipeline (table creation → insert → this route → browser), say so and I'll give you a quick way to seed one or two todos first.

Turn 5 — Debugging

You:

I only see this [HMR] connected in the console

Claude:

That [HMR] connected message is unrelated to your API route — it's just Next.js's dev server confirming its hot-reload websocket connection. You'll see that in the console on every page in dev mode, it's totally normal and not an error.

It sounds like you might be looking at the **browser DevTools console** (the JS console tab), rather than the actual page content. Here's how to check properly:

1. Make sure you're on the right tab/URL In your browser's address bar (not devtools), navigate to:

`http://localhost:3000/api/todos`

Press Enter and look at the **page itself** — not the console. You should see raw JSON text on the page, like `[]` or a list of todo objects.

2. If the page is blank or shows an error Open DevTools (F12) and check the **Network tab**, not Console. Look for a request to `todos` and check its status code:

- **200** — working, click it and check the “Response” or “Preview” tab to see what JSON came back
- **404** — the route file isn’t in the right place or named correctly
- **500** — an error in your code (likely a database issue) — check the terminal where `npm run dev` is running for a stack trace

3. Also check your terminal Since Route Handlers run server-side, any errors (e.g. db import failing, table not existing) will print in the terminal running `npm run dev`, not in the browser console.

Can you tell me:

1. What you see when you go to `http://localhost:3000/api/todos` directly in the address bar (the page content itself)?
2. Anything unusual in the terminal running `npm run dev`?

Turn 6 — Debugging

You:

the page shows the default content in `page.tsx`

Claude:

That means the request isn’t reaching your route file — Next.js is falling back to matching `app/page.tsx` instead, which happens when the folder/file structure for the API route isn’t quite right.

Let’s check the actual file layout. Run this from your project root:

```
find app -type f -name "*.ts" -o -type f -name "*.tsx"
```

Or simpler, just list the app folder:

```
ls -R app
```

What I’m checking for:

1. **Correct path:** it must be `app/api/todos/route.ts` — not `app/api/todos.ts`, not `app/api/todos/route.tsx`, not `app/todos/route.ts`. The folder structure matters exactly:

```
app/
  api/
    todos/
      route.ts
  page.tsx
  layout.tsx
```

2. **Filename is exactly `route.ts`** (or `.js`) — this is a reserved, case-sensitive filename Next.js looks for. `Route.ts`, `routes.ts`, `index.ts` won't work.
3. **Dev server picked up the new file** — Next.js usually hot-reloads new routes automatically, but occasionally a brand new route file needs a restart. Stop the dev server (Ctrl+C in the terminal) and run `npm run dev` again.

Can you paste the output of `ls -R app` (or just describe the folder structure you see in your editor's file explorer)? That'll tell us exactly what's wrong.

Turn 7 — Debugging

You:

Could not find a declaration file for module 'better-sqlite3'. Could that be the issue?

Claude:

That's a separate, non-blocking issue — it's TypeScript complaining it can't find type definitions for the package, but the code still works at runtime (Node doesn't care about TypeScript types, only your editor/compiler does). It wouldn't cause the page to fall back to `page.tsx` content, so it's not the cause of what you're seeing, but worth fixing anyway so your editor stops nagging you.

Fix it:

```
npm install --save-dev @types/better-sqlite3
```

This installs a separate package containing just the type definitions for `better-sqlite3` (the library itself is written in JS, so types are maintained separately). Once installed, the red squiggly under the import should disappear.

Back to the main issue — the route not being found. Can you paste the output of:

```
ls -R app
```

That'll show us the actual folder structure so we can see whether `app/api/todos/route.ts` exists where Next.js expects it.

Turn 8 — Debugging

You:

(pasted the output of `ls -R app`, showing `app/api/todos/route.ts` present)

Claude:

That structure looks correct — `app/api/todos/route.ts` is exactly where Next.js expects it. So the file itself isn't the problem. A few things to check:

1. Restart the dev server Next.js sometimes doesn't pick up brand-new route files with hot-reload alone. In the terminal running `npm run dev`:

- Press `Ctrl+C` to stop it
- Run `npm run dev` again
- Then reload `http://localhost:3000/api/todos` in the browser

2. Double check the exact URL Make sure you're going to:

`http://localhost:3000/api/todos`

not `/todos` (missing `/api`) or with a trailing slash issue.

3. Paste the contents of your `route.ts` Just to rule out a syntax error that's silently failing.

4. Check the terminal for errors After restarting and reloading the page, look at the terminal running `npm run dev` — any red error text, especially anything mentioning `todos` or `db`?

One more note: I see your project is inside a **OneDrive-synced folder**. This is a common source of odd, hard-to-diagnose issues with Node projects — OneDrive can lock files mid-write or delay syncing `node_modules`, which sometimes causes dev servers to behave inconsistently. It's not necessarily your issue here, but if problems keep popping up for no clear reason, moving the project outside OneDrive is worth trying.

Let's start with the restart + terminal check first, though — that's the most likely fix.

Turn 9 — Debugging

You:

im just going to localhost 3000

Claude:

That's the issue right there — `localhost:3000` on its own loads `app/page.tsx` (the homepage). Your API route lives at a different path, so you need to go to:

`http://localhost:3000/api/todos`

Type that full URL into your browser's address bar (or click into the address bar and add `/api/todos` after `3000`), then press Enter. You should see JSON output — likely `[]` if your `todos` table is empty.

Give that a try and let me know what shows up.

Turn 10 — Debugging

You:

oh - it works then

Claude:

Nice, that confirms the whole chain is working — Next.js routing, your `db.ts` connection, and the SQLite table all talking to each other correctly.

You're currently probably seeing `[]` since the table's empty. Next logical steps for lab 1:

1. **POST** `/api/todos` — add a new todo (insert into the table)
2. **PATCH/PUT** `/api/todos/[id]` — toggle a todo's completed status
3. **DELETE** `/api/todos/[id]` — remove a todo
4. Then wire up the **frontend** (`page.tsx`) to fetch and display todos, with a form and buttons calling these routes

Want to move on to the POST route next, so you can actually add todos and see something other than `[]`?

Turn 11 — Planning

You:

am I just naming them by what kind of operation it is?

Claude:

Good question — and the naming is actually a bit more specific than “name by operation.” Here's the pattern:

The file/folder name = the URL path, not the operation

The **folder structure** determines the URL, and the **function name inside** `route.ts` determines which HTTP method it responds to. So you don't create separate files per operation — you create one `route.ts` per URL, and inside it you export multiple functions named after HTTP methods.

Example:

```
app/api/todos/route.ts      → URL: /api/todos
app/api/todos/[id]/route.ts → URL: /api/todos/1, /api/todos/2, etc.
```

In `app/api/todos/route.ts` (operating on the whole collection):

```
export async function GET() { ... } // list all todos
export async function POST() { ... } // create a new todo
```

In `app/api/todos/[id]/route.ts` (operating on one specific todo — note the square brackets, that's a **dynamic route segment**):

```
export async function PATCH() { ... } // update one todo (e.g. toggle complete)
export async function DELETE() { ... } // delete one todo
```

Why split it this way?

This follows a REST convention: the **URL identifies the resource** (all todos, vs. one specific todo), and the **HTTP method identifies the action** (get/create/update/delete). So:

Method	URL	Meaning
GET	<code>/api/todos</code>	fetch all todos
POST	<code>/api/todos</code>	create a new todo
PATCH	<code>/api/todos/1</code>	update todo with id 1
DELETE	<code>/api/todos/1</code>	delete todo with id 1

Notice GET and POST share the same file (`/api/todos`) because they both act on the collection as a whole. PATCH and DELETE share a different file (`/api/todos/[id]`) because they both need to know *which* todo they're acting on — that's what `[id]` captures from the URL.

The `[id]` part

That's Next.js's syntax for a dynamic segment — whatever value shows up in that spot in the URL (e.g. the 1 in `/api/todos/1`) gets passed into your function so you can use it in your SQL query (`WHERE id = ?`).

Want me to build the POST route next, then the `[id]/route.ts` file with PATCH and DELETE together?

Turn 12 — Code Generation

You:

generate the rest of the todos

Claude:

Let's build out the rest.

1. Update `app/api/todos/route.ts` — add POST

```
import { NextResponse } from 'next/server';
import db from '@lib/db';

export async function GET() {
```

```

    const todos = db.prepare('SELECT * FROM todos ORDER BY id DESC').all();
    return NextResponse.json(todos);
}

export async function POST(request: Request) {
    const body = await request.json();
    const { title } = body;

    if (!title || typeof title !== 'string' || title.trim() === '') {
        return NextResponse.json(
            { error: 'Title is required' },
            { status: 400 }
        );
    }

    const result = db
        .prepare('INSERT INTO todos (title, completed) VALUES (?, 0)')
        .run(title.trim());

    const newTodo = db
        .prepare('SELECT * FROM todos WHERE id = ?')
        .get(result.lastInsertRowid);

    return NextResponse.json(newTodo, { status: 201 });
}

```

Notes:

- `request.json()` parses the JSON body sent by the frontend (e.g. `{ "title": "Buy milk" }`)
- Basic validation rejects empty titles with a `400` status
- `.run()` executes an insert/update/delete and returns metadata, including `lastInsertRowid` — the id SQLite just assigned
- We fetch and return the newly created row so the frontend has the full object (including its new id) without a second request
- `201` is the conventional status code for “created”

2. Create `app/api/todos/[id]/route.ts` — PATCH and DELETE

```

import { NextResponse } from 'next/server';
import db from '@/lib/db';

export async function PATCH(
    request: Request,
    { params }: { params: Promise<{ id: string }> }
) {
    const { id } = await params;
    const body = await request.json();
    const { completed } = body;

```

```

const existing = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
if (!existing) {
  return NextResponse.json({ error: 'Todo not found' }, { status: 404 });
}

db.prepare('UPDATE todos SET completed = ? WHERE id = ?').run(
  completed ? 1 : 0,
  id
);

const updated = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
return NextResponse.json(updated);
}

export async function DELETE(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;

  const existing = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
  if (!existing) {
    return NextResponse.json({ error: 'Todo not found' }, { status: 404 });
  }

  db.prepare('DELETE FROM todos WHERE id = ?').run(id);

  return NextResponse.json({ success: true });
}

```

Notes:

- `{ params }` gives you access to the dynamic `[id]` segment from the URL — in Next.js 15 (App Router), `params` is a Promise, so you await it
- Both routes check the todo exists first and return 404 if not
- PATCH here just toggles completed
- DELETE doesn't need a body — the `id` in the URL is enough

Also built a Server Component (`app/page.tsx`) and Client Component (`app/ToDoList.tsx`) to fetch and display todos, with a form and buttons wired to these routes via `fetch()`.

Turn 13 — Code Generation

You:

change the delete task feature into archiving so • A task cannot be deleted, only archived, so that it remains viewable.

Claude:

Good instinct — this is a common real-world pattern (soft delete). Instead of actually removing rows, we'll add an archived flag and filter on it.

1. Updated lib/db.ts schema:

```
import Database from 'better-sqlite3';

const db = new Database('todos.db');

db.exec(`
  CREATE TABLE IF NOT EXISTS todos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    completed INTEGER DEFAULT 0,
    archived INTEGER DEFAULT 0
  )
`);

const columns = db.prepare(`PRAGMA table_info(todos)`).all() as { name:
string }[];
const hasArchived = columns.some((col) => col.name === 'archived');
if (!hasArchived) {
  db.exec(`ALTER TABLE todos ADD COLUMN archived INTEGER DEFAULT 0`);
}

export default db;
```

2. app/api/todos/route.ts — GET filters WHERE archived = 0; POST inserts with archived = 0.

3. app/api/todos/[id]/route.ts — replaced DELETE entirely with a flexible PATCH that can toggle completed and/or archived:

```
import { NextResponse } from 'next/server';
import db from '@lib/db';

export async function PATCH(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const body = await request.json();

  const existing = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
  if (!existing) {
```

```

    return NextResponse.json({ error: 'Todo not found' }, { status: 404 });
  }

  const updates: string[] = [];
  const values: (number | string)[] = [];

  if ('completed' in body) {
    updates.push('completed = ?');
    values.push(body.completed ? 1 : 0);
  }
  if ('archived' in body) {
    updates.push('archived = ?');
    values.push(body.archived ? 1 : 0);
  }

  if (updates.length === 0) {
    return NextResponse.json(
      { error: 'No valid fields to update' },
      { status: 400 }
    );
  }

  values.push(id);
  db.prepare(`UPDATE todos SET ${updates.join(', ')} WHERE id = ?`).run(
    ...values
  );

  const updated = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
  return NextResponse.json(updated);
}

```

Notice there's no DELETE export anymore — the route simply doesn't support that HTTP method, matching the requirement that a task genuinely cannot be deleted.

4. app/TodoList.tsx — replaced the Delete button with an Archive/Unarchive button, and added a “Show archived tasks” checkbox that filters the already-fetched list client-side. Archived tasks are greyed out and disabled from further checking, but remain visible when the checkbox is ticked.

Turn 14 — Debugging

You:

Cannot find module './TodoList' or its corresponding type declarations.

Claude:

That error means TypeScript can't locate `TodoList.tsx` at the path you're importing from. Likely causes, in order:

1. The file wasn't actually saved as `TodoList.tsx` in the app folder
2. Wrong extension — it must be `.tsx` (not `.ts`) since it contains JSX
3. Dev server / TS server needs a restart
4. Case sensitivity mismatch between the filename and the import

Can you run `ls app` and paste the output? That'll tell us immediately which of these it is.

Turn 15 — Debugging

You:

(pasted `ls app` output confirming `TodoList.tsx` exists in the correct location)

Claude:

That confirms it: `TodoList.tsx` exists, right where it should be, correctly named. So the file itself is fine — this is a stale TypeScript/editor cache issue, which is common right after creating a brand new file.

Try these in order:

1. **Restart the dev server** — `Ctrl+C`, then `npm run dev` again.
2. **Restart the TypeScript server in your editor** (VS Code) — Command Palette (`Ctrl+Shift+P`) → **"TypeScript: Restart TS Server"**. This is usually the actual fix.
3. **If still showing** — close and reopen the file/tab, or reload the editor entirely.

Try step 2 first — that's the one that fixes this 9 times out of 10.

Turn 16 — Debugging

You:

it has an issue with the archived variable *(pasted `TodoList.tsx`, which referenced `todo.archived` throughout)*

Claude:

Good catch by the TypeScript checker — this is a type definition mismatch, not a logic bug. Your `Todo` type in `page.tsx` only listed `id`, `title`, and `completed`, but `TodoList.tsx` uses `todo.archived` in several places, which TypeScript doesn't recognise as a valid field.

Fix: add archived to the type in `page.tsx`:


```
export type Todo = {
  id: number;
  title: string;
  completed: number;
  archived: number;
};
```

That's it — one line. Since SQLite stores booleans as 0/1 integers, `archived: number` matches how `completed` is already typed, and the existing `!!todo.archived` conversions in `TodoList.tsx` will correctly turn that into a real boolean.

After adding that, restart the TS server again if the error doesn't clear immediately.

Turn 17 — Debugging

You:

im getting Runtime SyntaxError Failed to execute 'json' on 'Response': Unexpected end of JSON input (*error thrown from addTodo in TodoList.tsx at res.json()*)

Claude:

This means the response body from `/api/todos` (POST) came back empty — so `res.json()` has nothing to parse. That points to the server-side route throwing an error before it could send back a proper response, most likely in the POST handler.

First, check your terminal — there should be a stack trace right after you tried adding a todo.

Likely causes given recent changes:

1. Mismatch between the INSERT statement and the actual table columns (e.g. inserting `archived` when the column doesn't exist yet due to a migration issue)
2. The migration logic in `lib/db.ts` (the `PRAGMA table_info / ALTER TABLE` check) not having run correctly against an existing `todos.db`

Quick way to check the table's actual columns:

```
npx sqlite3 todos.db ".schema todos"
```

Paste that output along with the terminal error and I can tell you exactly what's mismatched.

Turn 18 — Debugging

You:

PATCH /api/todos/1 404 in 123ms (next.js: 32ms, application-code: 91ms) POST /api/todos 405 in 194ms (next.js: 166ms, application-code: 28ms) [browser]
unhandledRejection: SyntaxError: Failed to execute 'json' on 'Response': Unexpected end of JSON input at addTodo (app/ToDoList.tsx:20:31)

Claude:

That's helpful — two different problems, both about routing/method mismatches rather than the database:

1. POST /api/todos → 405 (Method Not Allowed)

This means Next.js found `app/api/todos/route.ts`, but it doesn't have a POST export it recognizes right now. Likely the dev server hasn't picked up the updated file.

Fix: stop the dev server (Ctrl+C) and restart it (`npm run dev`). Route handler changes sometimes need a full restart, not just hot-reload.

2. PATCH /api/todos/1 → 404 (Not Found)

This means Next.js can't find a route matching `/api/todos/1` at all — pointing to `app/api/todos/[id]/route.ts` either not existing yet, or being in the wrong place.

Check:

```
ls app/api/todos
ls "app/api/todos/[id]"
```

Can you run both and paste the output? Also restart the dev server regardless — that's very likely the fix for the 405 on its own.

Turn 19 — Debugging

You:

ok since adding the archive feature, the app is now broken. Cant add tasks anymore

Claude:

Let's get a clear picture of exactly what's on disk right now before guessing further. Can you paste the output of:

```
ls app/api/todos
cat app/api/todos/route.ts
```

And if the `[id]` folder exists:

```
ls "app/api/todos/[id]"
cat "app/api/todos/[id]/route.ts"
```

My guess is one of: the POST function got lost or malformed during edits, the `[id]` folder/file was never actually created, or a syntax error earlier in `route.ts` is silently breaking the whole file's exports.

Turn 20 — Debugging

You:

there's no `id` folder

Claude:

That's the core issue — without `app/api/todos/[id]/route.ts`, there's no route to handle `/api/todos/1`, hence the 404 on PATCH.

1. Create the folder and file:

```
mkdir "app/api/todos/[id]"
```

(Quotes needed since `[` and `]` can be interpreted specially by the shell.)

Then create `app/api/todos/[id]/route.ts` with the PATCH handler (status/archived-toggle logic).

2. Double check the separate POST issue — asked for the current contents of `app/api/todos/route.ts` to confirm the POST export was actually present.

3. Restart the dev server after creating the new file.

Turn 21 — Debugging

You:

(pasted contents of `app/api/todos/route.ts` — which turned out to contain the PATCH handler code)

Claude:

Found it — this is the `[id]/route.ts` content (the PATCH handler), but it's sitting in `app/api/todos/route.ts` (the collection-level file). That explains everything: no GET/POST exports in this file (hence the 405), and this PATCH was intercepting `/api/todos` requests while `/api/todos/1` still had nowhere to go (hence the 404, since the `[id]` folder didn't exist yet either).

Fix: swap the contents — restored the original GET/POST code into `app/api/todos/route.ts`, and placed the PATCH code (unchanged, it was correct) into the newly created `app/api/todos/[id]/route.ts`. Then restart the dev server.

Turn 22 — Code Generation

You:

Add these features: The user can view their tasks as a list, sortable by topic, by status and by due date. • Each task has one of three statuses: Todo, In-Progress, Complete. These are fixed; they are not user customisable. • A task that is overdue must be indicated in some way, but not as a status. • If the application is restarted, all information persists

Claude:

Built this in layers: schema, API, then UI.

1. lib/db.ts — expanded schema with topic TEXT, status TEXT DEFAULT 'todo', and due_date TEXT, plus migration checks (PRAGMA table_info + ALTER TABLE) so an existing todos.db upgrades safely. status is a plain TEXT column; the three fixed values are enforced entirely at the application layer.

2. app/page.tsx — expanded the Todo type with a Status = 'todo' | 'in-progress' | 'complete' union type, plus topic and due_date fields.

3. app/api/todos/route.ts — POST now accepts topic and due_date; every new task is always created with status: 'todo', since the frontend has no way to pick a different starting status.

4. app/api/todos/[id]/route.ts — added a VALID_STATUSES = ['todo', 'in-progress', 'complete'] array; PATCH rejects any status value outside that list with a 400, which is the actual enforcement point for “fixed, not customisable.”

5. app/ToDoList.tsx — added:

- A sortKey dropdown (“Date added” / “Topic” / “Status” / “Due date”) driving a sortTodos() helper
- A status <select> per task, populated only from the fixed STATUSES array
- An isOverdue() helper (due date in the past, and status isn’t complete) rendering a separate red “⚠ Overdue” badge next to the task — kept entirely separate from the status field, so overdue is a visual indicator, not a fourth status
- Persistence requires no extra work here — SQLite already writes to todos.db on disk with every insert/update, independent of the Node process, satisfying the restart-persistence requirement

Recommended deleting the existing todos.db and letting it regenerate fresh given how many columns changed, rather than relying on the migration path for test data.

Turn 23 — Code Generation

You:

Help generate the following:

- The repository must contain at least three tests that exercise real behaviour, runnable by a single documented command.
- The repository must contain documentation, as markdown, specifying:
 - Third-Party Code — the libraries and packages you installed, and one line on why each was chosen.
 - Database Design — the tables and the relationships between them.
 - Running It — the Node version and the exact commands to install, run and test the application, such that a reader can start it from a clean clone with nothing else to hand.

Claude:

1. Installed Vitest as the test runner (`npm install --save-dev vitest`) — lightweight, native TypeScript support, no browser environment needed since the routes are just server-side functions.

2. Made the database testable — changed `lib/db.ts` to read `process.env.DB_PATH` || `'todos.db'`, so tests can point at SQLite's in-memory mode (`:memory:`) without touching the real `todos.db`.

3. Added `vitest.config.ts` — sets `DB_PATH=':memory:'` for the test environment and mirrors the `@/*` path alias from `tsconfig.json`.

4. Wrote `tests/todos.test.ts` and `tests/todos-id.test.ts` — seven tests total, calling the actual Route Handler functions (GET, POST, PATCH) directly with real Request objects:

- POST creates a todo with default status 'todo'
- POST rejects an empty title
- GET returns created todos
- PATCH updates status to a valid value
- PATCH rejects an invalid status value
- Archiving a task hides it from GET but keeps it in the database
- PATCH returns 404 for a non-existent todo

5. Added a test script to `package.json`:

```
"test": "vitest run"
```

(`vitest run` rather than plain `vitest`, so it runs once and exits rather than entering watch mode.)

6. Generated `DOCUMENTATION.md` with three sections:

- **Third-Party Code** — a table listing next, react/react-dom, typescript, better-sqlite3, @types/better-sqlite3, tailwindcss, eslint, and vitest, each with a one-line reason for its inclusion.

- **Database Design** — full schema of the single todos table (id, title, topic, status, due_date, archived), with a note that status is enforced as one of three fixed values at the application layer rather than by a database constraint, and that archived implements the no-delete requirement instead of a real DELETE. Noted there are no relationships between tables, since the schema is single-table.
 - **Running It** — Node version requirement (18.18+, 20 LTS recommended), and the exact `git clone / npm install / npm run dev / npm test` sequence for a clean clone.
-

Turn 24 — Debugging

You:

(pasted a React hydration mismatch warning, all attributes named fdprocessedid) tasks are not being archived and im getting [the above]

Claude:

Two separate things — one harmless, one needs a closer look.

The hydration warning is not a bug in the code. Every mismatched attribute is `fdprocessedid="..."`, which isn't rendered anywhere in the app's code — that's the signature of a browser extension (typically a form-filler like Dashlane/LastPass) injecting attributes into inputs/selects/buttons after the page loads, which confuses React's hydration check. Confirmed by testing in an incognito window, where extensions are usually disabled by default.

Tasks not archiving — asked to check: any console/terminal error when clicking Archive; the Network tab status code for the `PATCH /api/todos/<id>` request; and a direct query of the database (`npx sqlite3 todos.db "SELECT id, title, archived FROM todos;"`) to see if the column is actually updating.

Turn 25 — Debugging

You:

(pasted browser console errors: multiple 404s on /api/todos/2, /api/todos/3, with Unexpected token '<' JSON parse errors, indicating an HTML error page was returned instead of JSON)

Claude:

That 404 (returning an HTML error page, hence the `<!DOCTYPE JSON` parse error) means Next.js can't find a route for `/api/todos/2`, `/api/todos/3`, etc. — same category of problem as before: the `[id]` dynamic route isn't registered.

Asked to confirm via:

```
ls "app/api/todos/[id]"
cat "app/api/todos/[id]/route.ts"
```

and to try restarting the dev server (`npm run dev`) regardless, since new/renamed route folders sometimes aren't picked up by hot-reload.

Turn 26 — Debugging

You:

Ok so the data persists but the archive button doesn't work and show archived tasks also doesn't work.

Claude:

Asked to pin down exactly what's currently running, given the file mix-ups earlier in the session:

```
cat "app/api/todos/[id]/route.ts"
cat app/ToDoList.tsx
```

Outlined what to check in each while waiting: for the archive button, the Network tab status/response for the PATCH request, and whether the UPDATE statement actually includes archived; for "Show archived", since it's pure client-side filtering with no network request, whether the checkbox's onChange is wired to setShowArchived and whether the actually maps over visibleTodos rather than the unfiltered todos.

Turn 27 — Debugging

You:

will these commands work in powershell?

Claude:

Confirmed both would work in PowerShell — cat is a built-in alias for Get-Content, and quoting the [id] path the same way as before would work the same way it did for the earlier ls commands.

Turn 28 — Debugging

You:

(pasted a PowerShell error: cat "app/api/todos/[id]/route.ts" → "An object at the specified path ... does not exist, or has been filtered by the -Include or -Exclude parameter.")

Claude:

That error is actually informative — PowerShell treats [and] as wildcard characters even inside quotes, which was misinterpreting the path.

Correct way to check with special characters in PowerShell:

```
Get-ChildItem -LiteralPath "app/api/todos"
```

Asked for that output to confirm whether the [id] folder was actually missing, or whether this was purely a PowerShell quoting quirk.

Turn 29 — Debugging

You:

(pasted Get-ChildItem -LiteralPath "app/api/todos" output, showing the [id] folder does exist) the ID folder definitely exists I can see it

Claude:

Good — it exists; the earlier cat error was just PowerShell misinterpreting [id] as a wildcard pattern, not a real missing-file issue. Gave the correct command to actually read it:

```
Get-Content -LiteralPath "app/api/todos/[id]/route.ts"
```

and cat app/ToDoList.tsx for the other file (no special characters, so plain cat is fine).

Turn 30 — Debugging

You:

(pasted the full contents of both app/ToDoList.tsx and app/api/todos/[id]/route.ts)

Claude:

Both files were actually correct — the PATCH handler properly handled archived, and the frontend's archiveTodo/showArchived logic was correctly wired, including that visibleTodos (not the unfiltered todos) was what got rendered in the .

Since the code was right, this pointed to a **stale build cache** — Next.js caches its route manifest in a .next folder, which may still have reflected an earlier, broken state from all the prior folder/file mix-ups.

Fix:

```
Remove-Item -Recurse -Force .next  
npm run dev
```


Then a hard refresh in the browser (Ctrl+Shift+R) to avoid a cached JS bundle too.

Turn 31 — Debugging

You:

it works now!

Claude:

Confirmed it was indeed the stale `.next` build cache holding onto the old, missing route. Noted for future reference: if a route seems to “exist in the code but 404s anyway,” deleting `.next` and restarting is a good first thing to try before chasing the code itself.

Turn 32 — Debugging

You:

should i be worried about this? *(pasted git add . output showing warning: in the working copy of 'app/page.tsx', LF will be replaced by CRLF the next time Git touches it and similar for package-lock.json and package.json)*

Claude:

Not worried — this is Git noting a line-ending difference between Windows (CRLF) and Unix-style (LF) files, extremely common and not a bug; nothing is broken or lost.

Not strictly necessary to fix for a solo lab project, but offered a clean fix if desired — adding a `.gitattributes` file with `* text=auto` to normalise line endings consistently regardless of OS, which also avoids messy diffs if the project is ever cloned on a different machine. Otherwise, safe to proceed with `git add . / commit / push` as normal.

Turn 33 — Code Generation

You:

any subtle UI change I can make to make it better? Nothing crazy

Claude:

Suggested five low-effort, isolated changes:

1. **Visual priority for overdue tasks** — give the whole task item a subtle red-tinted border/background when overdue, rather than relying on a small red label alone:

```
className={`... ${
  todo.archived ? 'bg-gray-100' : overdue ? 'border-red-300 bg-red-50' : ''
}`}
```

2. **Status color coding** — a colored left border per status (grey/yellow/green for todo/in-progress/complete) for faster scanning than reading each dropdown.
3. **Empty state** — a small “No tasks yet — add one above.” message when the list is empty, rather than blank space.
4. **Task count** — a small counter showing active task count near the top.
5. **Button hover states** — added hover: Tailwind classes to the Add and Archive buttons for better interactivity feedback.

Offered to fold all of these into a full updated `TodoList.tsx` if wanted, noting each is safe and isolated enough to add one at a time.

Turn 34 — Code Generation

You:

im not sure where to add point 2

Claude:

Clarified the two places needed: the constant definition near the other status constants —

```
const STATUS_COLORS: Record<Status, string> = {
  todo: 'border-l-4 border-l-gray-300',
  'in-progress': 'border-l-4 border-l-yellow-400',
  complete: 'border-l-4 border-l-green-400',
};
```

— and merging it into the `<li>`'s `className`, alongside the existing archived/overdue logic:

```
<li
  key={todo.id}
  className={`flex flex-col gap-2 border rounded px-3 py-2 sm:flex-row
sm:items-center ${STATUS_COLORS[todo.status]} ${
    todo.archived ? 'bg-gray-100' : overdue ? 'border-red-300 bg-red-50' : ''
  }}
>
```

Noted `STATUS_COLORS[todo.status]` is its own piece in the template string, separate from the archived/overdue ternary, since status color and archived/overdue background are independent signals that can both apply at once.

Turn 35 — Planning

You:

did you include the schema for the database as well?

Claude:

Confirmed yes — the `DOCUMENTATION.md` generated earlier includes a full Database Design section covering the `todos` table schema (`id`, `title`, `topic`, `status`, `due_date`, `archived`), each column's type, and notes on what each enforces (e.g. `status` restricted to three values at the application layer; `archived` implementing the no-delete requirement). Also notes there's only one table and no relationships between tables.

Double-checked that this section was written after the `sorting/status/due-date/overdue` features were added, so it reflects the schema as it stood at that point, and suggested the user cross-check it against the current `lib/db.ts` themselves in case of any manual tweaks made since, offering to regenerate it fresh if needed.

AI Declaration: The preceding document was reviewed and edited with: Claude-Web[Claude Sonnet 5]